A Comparative Study of Simple Neural Networks, BERT Fine-tuning, Domain-Adaptive Pre-training, and Large Language Model Prompting

Tiana Collingwood
School of Computer Science and Information Technology
Luzern University of Applied Sciences and Art
Rotkreuz, Switzerland
tiana.collingwood@stud.hslu.ch

Link to GitHub Repository: https://github.com/tccmc8/NLP_Project.git

***Abstract*— This paper presents a comparative study of text classification on the 20-Newsgroups dataset, a benchmark of approximately 18,000 Usenet posts distributed across 20 topically distinct categories. Five methodological approaches are evaluated in sequence: a shallow feedforward neural network with static word embeddings (Word2Vec, GloVe) and TF-IDF vectorisation; three BERT fine-tuning configurations ranging from a frozen linear probe to full parameter updates; task-adaptive masked language model pre-training followed by classification fine-tuning; zero-shot and one-shot-per-class prompting with Llama-3-8B; and parameter-efficient fine-tuning of Llama-3.2-1B via QLoRA. Results range from 14.4% accuracy under undertrained Word2Vec embeddings to 73.6% under MLM-adapted BERT fine-tuning, with TF-IDF (68.7%) outperforming all dense embedding baselines despite requiring no neural architecture. The experiments demonstrate that task-specific adaptation consistently outweighs model scale: Llama-3-8B without fine-tuning (50.0%) underperforms a decades-old statistical method, while even minimal gradient-based adaptation via QLoRA yields a 14 percentage point gain over prompting alone. Together, the results provide a controlled comparison of representational and computational trade-offs across the dominant NLP paradigms of the past decade.**

Keywords—text classification, 20-Newsgroups, Word2Vec, TF-IDF, GloVe, BERT, masked language modelling, domain-adaptive pre-training, Llama-3, zero-shot learning, few-shot learning, QLoRA, natural language processing, deep learning

I. Introduction

The purpose of this report is to solidify the learnings from the module Natural Language Processing (NLP) through experimenting with different text classification models, which progress chronologically. Text classification is one of the foundational issues that NLP addresses, classifying natural language text into predefined categories, these categories could be used for spam filtering, sentiment analysis and more. Despite the multitude of methods and research into NLP, there is no single approach however some approaches work better in certain circumstances: methods based on term-frequency statistics remain competitive on keyword-driven corpora, while transformer-based models [1] and large language models (LLMs) [2][3] offer richer representations at substantially greater computational cost. This paper presents a comparative study of five distinct approaches to multi-class text classification on the 20-Newsgroups dataset [4], a canonical benchmark of approximately 18,000 Usenet posts distributed across 20 topically distinct categories. The report begins with a simple two-layer neural network paired with Word2Vec [5][6], TF-IDF, and pre-trained GloVe embeddings [7], then progressing through BERT fine-tuning [8][9], domain-adaptive masked language model pre-training [10], Llama-3 zero- and few-shot prompting [3][11][12], and a parameter-efficient approach, Quantised Low-Rank Adaptation (QloRA). This report evaluates how the choice of text representation and model architecture affects classification accuracy, generalisation, and practical trade-offs. The progression from A1 to Part E mirrors the historical arc of the field, moving from static embeddings fed into shallow feedforward networks, through progressively richer BERT fine-tuning strategies, to domain-adaptive MLM pre-training and finally to prompting and parameter-efficient adaptation of a 8B-parameter generative model. Each step addresses a specific limitation of its predecessor: static embeddings are superseded by contextualised BERT representations; full fine-tuning is made tractable through linear probing and partial freezing; the domain gap between general pre-training and the 20-Newsgroups corpus is narrowed via DAPT; and the prohibitive cost of fully fine-tuning Llama is overcome by QLoRA. Together, these experiments provide a controlled comparison across the most prominent NLP methodologies of the past decade.

A. Related Work

This section organises the relevant literature thematically across the models and experiments conducted in the report.

1) Word Representations

Classical text classification relied on sparse, high-dimensional representations such as TF-IDF, which encode token frequency relative to corpus-wide inverse document frequency:

$$TF(t,d) = \frac{count\ of\ term\ t\ in\ document\ d}{total\ number\ of\ terms\ in\ document\ d} \quad (1)$$

$$IDF(t,D) = \log\left(\frac{total\ number\ of\ documents\ |D|}{number\ of\ documents\ containing\ term\ t\ in\ |D|}\right)$$

$$TF-IDF(t,d,D) = TF(t,d) \times IDF(t,D)$$

Equation (1) shows the breakdown on TF-IDF. TF takes the term frequency, how frequently the term, *t*, appears in a document, *d*, in the corpus, *D*. IDF measures how common or rare the term, *t*, is across the corpus, *D*, penalising more frequent words [8]. TF-IDF helps to understand how relevant words or terms are in the corpus but it does not describe the relationship between words as the vectors are orthogonal by design so like words do not group together in the vector space, sharing proximity [9]. As it is computationally cheap and interpretable, this report uses TF-IDF as a control experiment

/ critical baseline to determine whether a more complex neural network (NN) and embedding is truly necessary.

TF-IDF being unable to determine relationships between words and their meaning, a shift began to dense, low-dimensional word embeddings [10] like Word2Vec and GloVe. Both models utilise a simple NN which is feedforward network (FFN) that updates its own parameters (weights and biases) with each new example, later fixed after training [11]. The preprocessing pipeline is the same across the models, it consists of: tokenisation, stopword removal, stemming or lemmatisation, POS tagging, and information retrieval [9]. In this report lemmatisation was chosen over stemming as the meaning in crucial and lemmatisation is better at picking up the meaning of the word e.g. with lemmatisation the root of 'better' would be 'good' whereas the root of 'better' using stemming would be 'better'. After preprocessing the tokens can then be run through the FFN which consists of two-layers:

$$FFN(x) = \max(0, x\,W1 + b1)\;W2 + b2 \qquad (2)$$

Equation (2) describes a position-wise FFN where *x* is the token / word inputted from the previous layer (input or another layer in the FFN), W1 is the first weight matrix which connects the input features to the first hidden layer, b1 is the first bias vector, W2 is compresses the matrix to its original size and b2 is the final set of learnable constants [9]. max() is the activation function which lays between sub-layers, also known as a ReLU activation [1]:

$$ReLU(z) = \max(0, z) \qquad (3)$$

Equation (3) processes the pre-activation input, z, and passing through positive inputs unchanged and negative inputs as 0, this helps the model learn complex patterns by introducing non-linearity [9]. For the input, Word2Vec or GloVe are used to translate text into numerical embeddings.

Word2Vec remains the canonical entry point to neural word embeddings [5]. The model learns representations by predicting a word from its surrounding context (Continuous Bag-of-Words) or predicting surrounding words from a target (Skip-Gram), such that words appearing in similar contexts are mapped to nearby vectors. [9] The model learns by training FFN using a sliding window over the training corpus, seeing which words appear together in which contexts [9]. A key practical consideration is the number of training epochs: embeddings trained for a single epoch over a small corpus capture less distributional signal than those trained for 20 epochs, motivating the comparison explored in the methodology.

GloVe learns word embeddings by analysing how often words appear together across an entire corpus [7], rather than through a sliding window like Word2Vec, producing vectors with similar semantic properties; such as 'king - man + woman = queen' [9]. GloVe is grounded in global word co-occurrence counts rather than local context prediction like Word2Vec.

A critical limitation shared by both Word2Vec and GloVe is that they are static: each word type receives a single vector regardless of context. Polysemous words (e.g., bank as a financial institution vs. a riverbank) are represented by a single average embedding, losing contextual disambiguation. This limitation drove the subsequent shift to contextualised representations.

2) Tranformers

The dominant backbone underpinning all contextualised models discussed in this report is the Transformer. The architecture relies on an attention mechanism to find the dependencies between the input and the output [1]. The central operation, Scaled Dot-Product Attention, computes:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \qquad (4)$$

Equation (4) shows the attention mechanism that is used to guide the AI to focus on the most relevant parts of the sentence, this is does through (4). The *Key's* (all the other terms in the sentence) relevance to the *Query* (word in focus) is done through *QK^TI.* To stabilise the gradient during training, the score is divided by the root of the *key* dimension, then passed through softmax to be converted into weights and only the high weights are passed forward as the weights are multiplied by *V* (meaning of the word).

Then the encoder processes input tokens bidirectionally, while each encoder layer additionally contains a position-wise feed-forward network of the form of (2) [1]. Crucially, the self-attention mechanism reduces the maximum path length between any two positions for recurrent layers [9], enabling far more effective learning of long-range dependencies [1]. It is this architectural property that makes Transformers the foundation of every modern pre-trained language model.

3) BERT

BERT (Bidirectional Encoder Representations from Transformers) was designed to pre-train bidirectional representations taking context from both the left and the right [12]. This contrasts with earlier unidirectional models, a restriction that is sub-optimal for classification tasks requiring full-sentence context.

BERT is pre-trained using two objectives. The Masked Language Model (MLM) randomly replaces 15% of input tokens with a [MASK] token and trains the model to predict the original vocabulary item from its bidirectional context.

The Next Sentence Prediction (NSP) task additionally trains the model to predict whether two segments are consecutive. The standard BERT-base configuration comprises L=12 Transformer blocks, H=768, and A=12, totaling approximately 110M parameters [12] with L being the number of layers, H the hidden size that dictates the amount of information passed to the next layer, A – the number of attention heads run in parallel, and the parameters that are accumulated across each layer. For downstream classification, not from scratch, BERT appends a task-specific head to the token representation and fine-tunes all parameters end-to-end.

This report investigates three fine-tuning strategies for BERT: (1) full fine-tuning of all 109M parameters, (2) a linear probe (frozen BERT encoder, head only), and (3) partial freezing of the top two encoder layers plus the head. The contrast between these conditions directly probes the extent to which BERT's pre-trained representations are general-purpose versus task-specific.

4) Domain-adaptive pre-training

A key insight motivating BERT Classification Task of this report is that general-purpose pre-training may leave significant domain-specific signal unexploited. There are two complementary strategies to combat this: Domain-Adaptive

Pre-Training (DAPT), which continues MLM pre-training on large in-domain corpora, and Task-Adaptive Pre-Training (TAPT), which pre-trains on the (much smaller) unlabelled task dataset itself [13]. For tasks in the biomedical and computer science domains DAPT yields the largest gains (e.g., ACL-ARC F1 improves from 63.0 to 75.4) [Gururangan et al., 2020, Table 3].

TAPT is found to be surprisingly competitive with DAPT despite using a far smaller corpus [13], which is beneficial for this report as the dataset is not particularly large. This approach will be taken and BERT will be further pre-trained on the 20-Newsgroups corpus before classification fine-tuning, TAPT.

5) Tokenisation

Unlike the preprocessing process used for word representation models, BERT and Llama use their own tokenisation strategy. BERT employs WordPiece tokenisation with a 30,000-token vocabulary [12], the tokenisers break down text into subword units [9]. WordPiece decomposes out-of-vocabulary words into known subword units [9], enabling the model to handle morphologically rich text without a large vocabulary.

Llama, by contrast, employs Byte-Pair Encoding (BPE), which is described by Hugging Face documentation as the "most popular tokenization algorithm in Transformers, used by models like Llama" [14]. BPE iteratively merges the most frequent symbol pair in the training corpus, producing a compact vocabulary that decomposes rare and out-of-vocabulary words into common substrings where common words stay as single tokens and rare words become subwords, representing unknown words [9]. The choice of tokeniser has downstream effects on out-of-domain generalisation, a consideration relevant to applying Llama to the 20-Newsgroups domain.

6) LLMs and In-Context Learning

Whereas BERT is an encoder model producing bidirectional representations suited to classification, autoregressive decoder models such as the GPT family operate differently; generating text token by token, left to right. Brown et al. (2020) scaled this paradigm with GPT-3, demonstrating that large language models can perform tasks through in-context learning alone, without any gradient updates [2]. This finding motivates the prompting experiments in Methodology D , where Llama-3, a GPT-class decoder model, is evaluated on 20-Newsgroups classification purely through prompt design (few-shot and zero-shot). In the book *Hands-on large language models: language understanding and generation* Alammar and Grootendorst define the two prompting regimes applied in Methodology D: zero-shot prompting provides no examples, while few-shot prompting supplies labelled instances directly in the prompt context [9].

7) QLoRA

Full fine-tuning of large language models requires updating all parameters and storing full-precision gradients and optimiser states, making it computationally prohibitive for 7B+ parameter models on consumer hardware. Low-Rank Adaptation (LoRA) addresses this by freezing the pre-trained weights and injecting trainable low-rank decomposition matrices [9].

QLoRA extends LoRA by quantising the frozen base model weights to 4-bit NormalFloat (NF4) precision, enabling fine-tuning of models such as Llama-3 on a single GPU [15]. As described in practical implementations of Llama-3.1 fine-tuning, QLoRA allows practitioners to "get started with the new Llama models and customize Llama-3.1-8B" for downstream tasks such as classification without the memory requirements of full fine-tuning [16]. The combination of 4-bit quantisation with 16-bit LoRA adapters yields minimal accuracy degradation relative to full-precision fine-tuning while reducing GPU memory requirements by approximately 4×. This is the technique employed in Methodology E of this report to adapt Llama-3 for the 20-Newsgroups classification task.

II. Methodology

This section explores a shallow neural network, Word2Vec, TF-IDF and GloVE, the two embedding models: Word2Vec and GloVe utilise a shared FFN architecture of two linear layers which ReLU in-between to solve the vanishing gradient issue; creating non-linear patterns that help the model learn more complex patterns. Along with the ReLU layers in-between, the FFN is trained with cross-entropy loss (3) to measure the difference between the true labels and the models labels. As mentioned in Word Representations, the preprocessing pipeline includes; lowercasing, regex tokenisation, stopword removal and lemmatisation, tagging, and information retrieval. It is important to note that the experiments are tracked in the notebook, grouped in a table after running each part (A to E) which tracks the per epoch training and validation loss, accuracy, and weight L2 norm evolution.

A. Simple Neural Network

The first model that was explored in this project was Word2Vec embeddings. Mean pooling is used to produce a single 100-dimensional document vector [17], capturing the overall semantic meaning. Training on the newsgroup corpus for 1 vs. 20 epochs was done and visualized in Figure 1. The vectors cluster together after extended training. This is further shown as at epoch 1 is nearly random (14.4% accuracy, F1=0.107) as the model has only seen the word in content once and the weight updates are small. Whereas at epoch 20 (58.8% accuracy, F1=0.571) semantic grouping has occurred as the context-window has stabilised and like words have been pulled closer together in the embedding space (Figure 1). With more passes, the model sees more context through co-occurrence, so the embeddings better reflect the semantic neighbourhood. For this report, the epochs are capped at 20 as it a relatively small dataset.

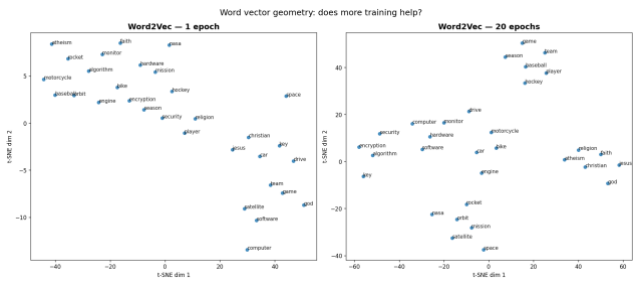

Figure 1 - t-SNE Visualisation of Word2Vec Embeddings: 1 Epoch vs. 20 Epochs

TF-IDF is then tested using 20,000-feature sparse vectorization and it substantially outperformed Word2Vec.

This could be due to several benefits of using TF-IDF (68.7% accuracy, F1=0.680); TF-IDF is lossless. It is lossless in terms of discriminative key words; it is not diluted by averages. One of the tradeoffs of TF-IDF is that the notion of synonyms does not exist, luckily topic terminology is consistent in the current dataset however, it could pose an issue in other larger datasets. Figure 2 shows the TF-IDF confusion matrix, which shows that domain specific frequently used classes like sports are rewarded by TF-IDF and are easier to classify. Groups with similar terminology on the other hand, like the comp.*cluster, is a problem area as there is a lot of overlapping vocabulary so the classifications can bleed into one another. Clearly, TF-IDF succeeds where topic vocabulary is *exclusive* to one category and fails where categories share a vocabulary domain. One of the main reasons that TF-IDF may outperform Word2Vec is that 20-Newsgroups is a very lexical, literal dataset where a lot of the terms are exclusive to a single class which is a bottleneck for mean-pooling that averages the vectors to a document vector, diluting the signal.

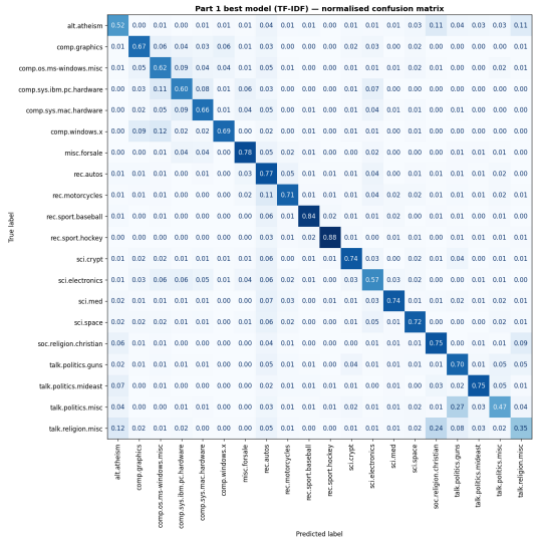

Figure 4 - Normalised Confusion Matrix, TF-IDF Best Model

An additional experiment in this section is pre-trained GloVe-wiki-gigaword-100 loaded via Gensim. Usually, GloVe has a larger external corpus which may create more reliable embeddings for technical vocabulary which TF-IDF struggled with, *.cluster. Using pre-trained with GloVe resulted in 57.2% accuracy and F1=0.552, slightly less than 20 epoch Word2Vec which was unexpected. Upon reflection, although GloVe was trained on Wikipedia text and with the current dataset, a lot of the vocabulary is domain specific and in this instance worked better than Word2Vec. Both models, GloVe and Word2Vec, suffer from the same mean-pooling bottleneck hence, TF-IDF performs better.

1) Best Model: TF-IDF

The best model is TF-IDF. Figure represents two side-by-side line charts tracking validation loss (left) and validation accuracy (right) across 30 training epochs for all four Simple NN experiments: Word2Vec 1 epoch, Word2Vec 20 epochs, TF-IDF, and GloVe. TF-IDF (green) converges fastest and highest, reaching ~76% validation accuracy within the first 5 epochs and plateauing there for the remainder of training, a sign that the model has extracted nearly all available signal

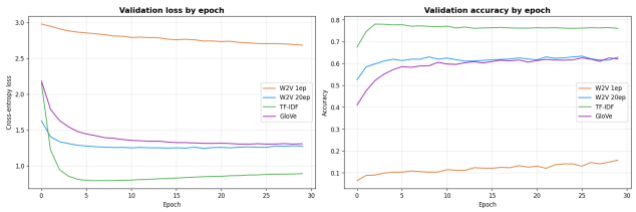

Figure 2 - Validation Loss and Accuracy Curves by Input Representation

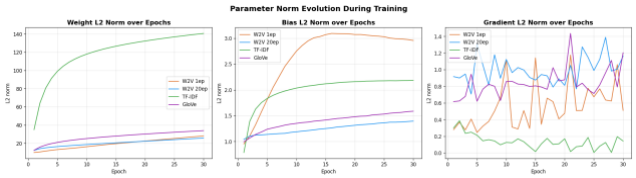

Figure 3 - Parameter Norm Evolution During Training

early. Word2Vec 20 epochs and GloVe track very closely together and converge to roughly the same ceiling (~62%), which tells you that the mean-pooling bottleneck, not the embedding quality, is limiting both of them equally. In terms of Simple NNs, embedding is not necessary as TF-IDF gets better results at a lower amount of epochs, as validation accuracy plateaus at epoch 5. Figure 5: The weight norm chart (left) is the most informative: TF-IDF's norm grows to ~140 while the dense embedding models stay below 30. This is a direct consequence of TF-IDF's 20,000-dimensional input, the first linear layer has 5.1M parameters and they all update, so the total weight magnitude is much larger. It does not mean TF-IDF is "more trained"; it reflects the difference in input dimensionality. The bias norm chart (centre) shows W2V 1 epoch reaching the highest bias values, a sign the model is essentially learning to predict the majority class rather than discriminating between classes, since its embeddings carry no useful signal. The gradient norm chart (right) shows W2V 1ep with the most erratic gradients (high variance, unstable), while TF-IDF gradients (green) are the smoothest and smallest by epoch 5, mirroring Figure 4.

Table 1 - Simple NN Model Comparison

Experiment	Input dim	Hidden	Weight params	Bias params	Total params	Weight L2	Bias L2	Grad L2	Test Acc	Macro-F1
W2V 1 epoch	100	256	30,720	276	30,996	27.948	2.956	0.510	0.1442	0.1069
W2V 20 epochs	100	256	30,720	276	30,996	25.669	1.395	1.163	0.5875	0.5712
TF-IDF (20k)	20k	256	5,125,120	276	5,125,396	140.608	2.182	0.143	0.6868	0.6798
GloVe pretrained	100	256	30,720	276	30,996	33.817	1.586	1.202	0.5722	0.5519

B. Fine-Tuning BERT

BERT-base tokenisation and Hugging Face Trainer API is experimented with in this section, particularly levels of trainability of the weights and biases of the mode. The metric used to assess the models is Macro-F1. Macro-F1 is used as it takes the average F1 score for each class, so there is equal class weighting across all 20 categories. As seen in Figure 5, Run A is a full fine-tuned BERT model where all the parameters are updateable, Run B is a frozen BERT model where none of the parameters are changeable, and Run C, a partial freeze, that has a small fraction of the parameters can be trained. The different levels of trainability makes the compute trade-off concrete and visual for example, Run C

trains 13.5% of Run A's parameters for 97% of the performance).

Run A updates all 109,497,620 parameters of BERT-base across 3 training epochs using a learning rate of 2e-5 and weight decay of 0.01. The learning rate is deliberately small, standard practice for transformer fine-tuning, to avoid catastrophic forgetting, the phenomenon where aggressive gradient updates overwrite the general linguistic knowledge encoded during BERT's pre-training on 3.3 billion tokens. The training curves (Figure 6) show Run A converging cleanly,

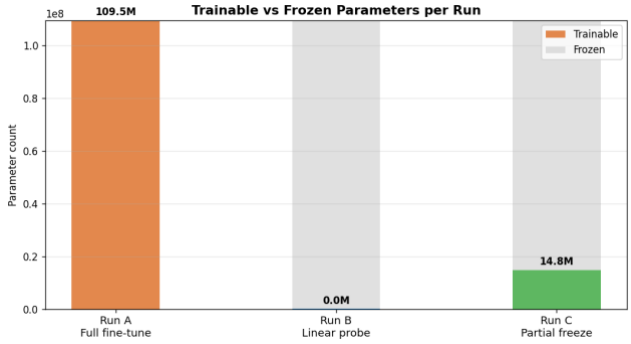

Figure 5 - Trainable vs. Frozen Parameters per BERT Run

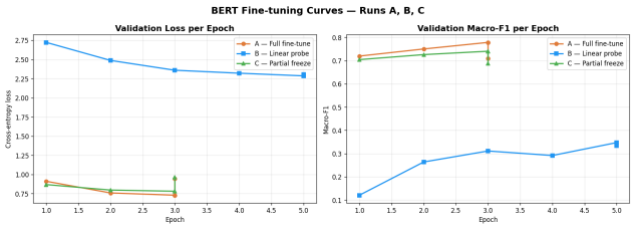

Figure 6 - BERT Fine-Tuning Training Curves: Runs A, B, C

with validation loss dropping from ~0.92 at epoch 1 to ~0.75 at epoch 3 and macro-F1 rising from 0.72 to 0.75. Run A achieves the best BERT result of 72.8% accuracy and macro-F1 of 0.709 — a 4.1 percentage point accuracy gain over the best Part A model (TF-IDF, 68.7%). The classifier head weight distribution (Figure 9) shows a tight Gaussian centred at zero (σ=0.0212), indicating the head made small, precise adjustments guided by gradient signal flowing back through all 12 encoder layers simultaneously. This is the performance ceiling for standard BERT fine-tuning in this project, and the baseline against which Runs B and C are measured.

Run B freezes all BERT parameters entirely and trains only the 15,380-parameter classification head, a 768×20 linear layer mapping the [CLS] token representation to one of the 20 newsgroup categories. The motivation for this experiment is diagnostic rather than practical: if BERT's pre-trained representations were inherently task-discriminative, a linear head alone should achieve competitive performance without any fine-tuning. The results refute this hypothesis clearly. Run B achieves only 38.7% accuracy and macro-F1 of 0.334, worse than TF-IDF (68.7%) and barely above Word2Vec at 20 epochs (58.8%). The training curves (Figure 7) show the linear probe's loss remaining above 2.3 across all 5 training epochs, while Runs A and C drop below 1.0 within the first epoch. The head weight distribution (Figure 9) is revealing: the head weights are more widely spread (σ=0.0545) with heavier tails than Run A, and the per-class biases show large variation across classes (some reaching ±0.015), the head is working at the limits of what a linear transformation can achieve over representations that were never optimised for this task. This result demonstrates that BERT's generic [CLS] representations, while rich in general linguistic structure, are not linearly separable across the 20-Newsgroups taxonomy without task-specific adaptation. Fine-tuning is not an optional enhancement, it is the mechanism that makes BERT useful for classification.

Run C takes a middle path: encoder layers 0–9 (out of 11) are frozen, while the top two encoder layers (10–11) and the classification head remain trainable, giving 14,843,660 trainable parameters, 13.5% of Run A's total, as shown in the parameter breakdown (Figure 8). The rationale draws on a well-established property of transformer models: lower layers encode general syntactic and morphological features that transfer broadly across tasks, while upper layers encode more task-specific, semantic features that benefit most from fine-tuning. By freezing the lower 10 layers, Run C preserves the general linguistic representations learned during pre-training while allowing the upper layers to adapt their attention patterns to the distributional properties of newsgroup text. The results support this design: Run C achieves 71.0% accuracy and macro-F1 of 0.688, only 1.8 percentage points below Run A in accuracy and 0.021 below in macro-F1, while training just 13.5% of the parameters. The training curves (Figure 7) show Run C tracking Run A almost identically across epochs 1–3 before marginally diverging at epochs 4–5, suggesting that the upper two layers capture most of the task-specific adaptation needed. The head weight distribution (Figure 9) closely resembles Run A (σ=0.0218 vs 0.0212), confirming that the classification head reaches a similar learned state in both runs. In a resource-constrained deployment scenario, where training time, compute cost, or memory are limited, Run C represents the most efficient configuration in this project: it recovers approximately 97% of Run A's macro-F1 at 13.5% of the parameter update cost.

1) Best Model: BERT Full Fine-tuning

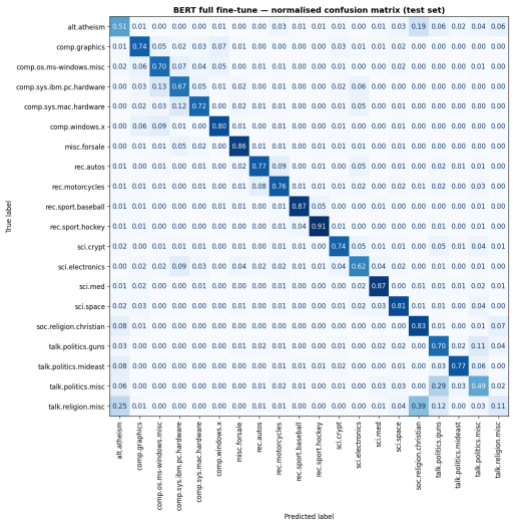

Figure 7 - Normalised Confusion Matrix, BERT Full Fine-Tune

Among the fine-tuned BERT models, the fully fine-tuned model had the best metrics that has overtaken TF-IDF. The already easy to classify sports accuracy (hockey and baseball) has increased. The *.comp cluster has meaningfully improved as BERT's contextual attention is better at distinguishing how computing terms are used even when the

vocabulary overlaps. Other contextually rich clusters, including medical language, benefited from using BERT. Despite the context aware nature of BERT, the model struggles with categories genuinely overlap in content; people discuss atheism on the religion group and politics group.

Compared to TF-IDF, BERT improves most on categories where *context* disambiguates shared vocabulary (computing subcategories, science), but the religion/politics cluster remains structurally hard because the confusion reflects genuine content overlap, not a model limitation.

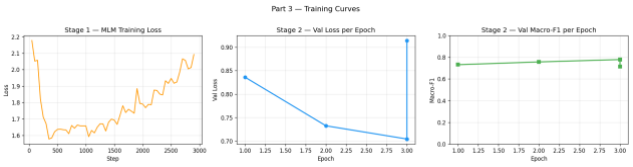

Figure 8 - Two-Stage Training Curves: MLM Pre-Training and Classification Fine-Tuning

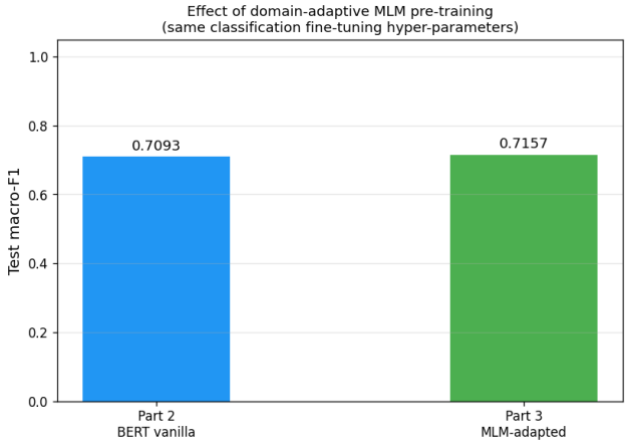

Figure 9 - Effect of Domain-Adaptive MLM Pre-Training on Test Macro-F1

C. BERT Classification Task

In this report, MLM pretraining is done on the 20-Newsgroup corpus using DataCollatorForLanguageModelling. Perplexity is used as a sanity check, making sure that MLM pretraining was successful; lower perplexity = better MLM predictions = more adapted to the domain. Pre-training is done on domain-specific text before fine-tuning consistently improves downstream performance, a part of TAPT. BERT Classification is made up of two tasks:

Task 1 MLM on dataset: continue BERT's masked language modelling objective on the newsgroup training text; dynamic masking (15% of tokens per step: 80% replaced with [MASK], 10% random, 10% unchanged); track training loss and perplexity.

Task 2 Classification fine-tuning: identical protocol to Run A (full fine-tune, lr=2e-5, 3 epochs); the only difference is the starting checkpoint.

This method has the best result across all parts, beating full fine-tuning (72.8%) by a small but consistent margin but does this make it a viable option for such a small amount of change? Stage 1 adds a full additional training phase; the 0.8% accuracy gain is modest; in a resource-constrained setting, Run A may be preferable.

Figure 8 shows the MLM loss chart (left) shows a characteristic pattern: rapid initial drop as the model quickly adapts its masked prediction to newsgroup vocabulary, followed by an increase and fluctuation around epochs 1,500–3,000. This rise is expected and not a sign of failure, as the model adapts to the domain distribution, it is effectively "forgetting" some of its general-purpose predictions in exchange for more domain-specific ones. The plateau and noise at later steps indicate the model has approximately converged on the newsgroup language distribution. The Stage 2 classification curves (centre and right) show clean convergence: loss drops consistently across all 3 epochs, and macro-F1 is already at ~0.75 from epoch 1, a notably strong starting point compared to Fine-tuning BERT's Run A, which started at ~0.72 at epoch 1. This higher starting F1 is the signature of the domain-adapted initialisation. Note the spike in Stage 2 validation loss at epoch 3, is worth flagging as a sign of possible slight overfitting at the tail end of fine-tuning.

D. Llama-3 for Few-shot and Zero-shot Learning

In order to run Llama-3, Groq API backend (Llama-3-8B-8192) was used, prompting engineering for constrained single-line output. This caused for a limited and long training process despite using GPU, Hugging Face tokens is a faster alternative which is also free. In sections A to C, there have been no gradient updates, no task-specific training, inference only; the model's parameters are fixed. The rationale behind prompt engineering in these experiments is the output format constraint, requiring Llama-3 to produce only the category name on a single line, was necessary to ensure responses could be deterministically parsed against the 20 fixed label strings, since without it the model's natural tendency toward verbose, conversational output would make label extraction unreliable. The two learning techniques used are:

- Zero-shot: system prompt only, no examples
- 1-shot per class (20 examples total): one labelled example per category in the prompt

Llama-3 was not trained on 20-Newsgroups; it has no prior exposure to the specific category taxonomy; its priors about what "comp.sys.ibm.pc.hardware" means come from general pre-training, which may conflict with the Usenet-specific category structure

Some categories are intuitively harder for a general LLM: the fine-grained comp.* subcategories share much vocabulary; without any training signal, the model has no basis for distinguishing them.

The three-tier label matcher applied exact string matching first, followed by case-insensitive partial matching, and finally fuzzy substring matching as a last resort; any response failing all three tiers was recorded as unparseable, yielding rates of 3.5% under zero-shot and 3.2% under few-shot conditions, a marginal difference suggesting that adding examples improved output formatting only negligibly.

In this project, the few-shot condition supplies one example per class (20 examples total), framing classification as in-context learning over the full 20-Newsgroups taxonomy. However, contrary to the expectation that few-shot examples improve consistency, the C results show no

meaningful gain; accuracy marginally decreases from 50.0% to 49.0%, while macro-F1 improves only slightly from 0.494 to 0.509. This suggests that for a 20-class structured taxonomy, a single example per class is insufficient to reliably anchor the model's output, and that the benefit of few-shot learning does not generalise straightforwardly to fine-grained multi-class settings [9].

Comparison with fine-tuned models: the 20+ percentage point gap between Llama-3 (50%) and MLM-adapted BERT (73.6%) demonstrates that scale alone does not replace task-specific training. Despite Llama-3-8B containing roughly 73 times more parameters than BERT-base, zero-shot classification achieves only 50.0% accuracy and macro-F1 of 0.494, 18.7 percentage points below TF-IDF, a decades-old statistical method, demonstrating that model scale without task-specific training is insufficient for structured, label-constrained classification, and that parameter count is a poor proxy for task performance.

It appears that zero-shots performs well on some of the classes as it seems that Llama-3 has seen a large amount of text related to classes like space exploration, hockey and Christianity which use very distinctive vocabulary that the model correctly associates with the right category. Zero-shot struggles with fine-grained categories like the *.comp cluster, which has consistently been troublesome, that have brand-specific vernacular and Llama-3 doesn't appear to have examples, so it defaults to a generic category.

Few-shots helps as it has format anchoring that tells the model that the answer should be exactly 'rec.sport.hockey' and not 'Hockey' which reduces unparseable outputs. By seeing an example of a class post, few-shot does lexical priming and disambiguation which reminds the model which vocabulary belongs to which model and defines boundaries of where a document falls between two similar categories.

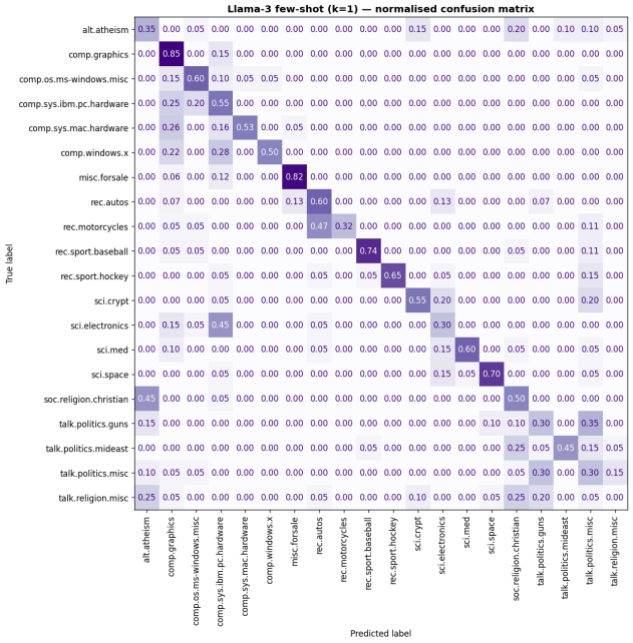

Figure 10 - Normalised Confusion Matrix, Llama-3 Few-Shot k=1

There is not much of a difference between zero-shot, few-shot and randomly classifying but between the two experiments, few-shot has a miniscule lead in the Macro-F1. Figure 10 depicts the matrix where a few classes are excellent (comp.graphics (0.85) and misc.forsale (0.82)) because theses categories are highly distinctive features that Llama-3's general training has encoded well. Unfortunately, the troublesome comp.* cluster collapses badly: comp.sys.ibm.pc.hardware drops to 0.55, comp.sys.mac.hardware to 0.53, comp.windows.x to 0.50, and mass confusion flows into comp.graphics. Llama-3 without task-specific training has no way to learn the fine-grained taxonomy of Usenet's computing subcategories. The religion/politics cluster is severely confused along with rec.motorcycles which leaks into rec.autos as Llama-3 conflates the two vehicle categories without the task-specific signal that separates them. The off-diagonal pattern is more scattered compared to A and B, where confusions were concentrated in predictable clusters; Llama-3's errors are less systematic, which suggests the model is making more arbitrary label choices when uncertain.

E. Fine-tuning using QLoRA

Parts A to D establish that Llama-3 without task-specific training lags substantially behind fine-tuned BERT, raising the question of whether parameter-efficient fine-tuning can close this gap while remaining tractable on consumer hardware. To investigate this, QLoRA [9] is applied to Llama-3.2-1B-Instruct, a model small enough to run on a Colab T4 GPU. The base model weights are stored in 4-bit NormalFloat (NF4) format via BitsAndBytes, reducing VRAM requirements from approximately 16GB in half-precision to 1.1GB, as shown in Figure 16. Crucially, quantisation applies only to the frozen base model weights; the LoRA adapter weights remain in full float32 precision and are the only parameters updated during training. Rather than modifying all 615.6M base model parameters, LoRA [X] inserts low-rank decomposition matrices, denoted A and B, into the attention layers of the network. Only these adapter parameters are trainable, amounting to 6.3M parameters, or 1.01% of the total, as shown in Figure 11.

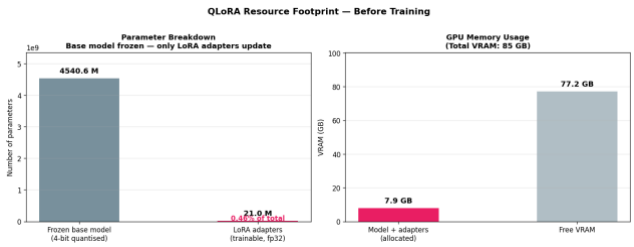

Figure 11 - QLoRA Resource Footprint: Parameter and VRAM Breakdown

Training follows an instruction-following format in which each newsgroup document is wrapped in the Llama-3 Instruct chat template and the model is trained to generate the correct category label as a text completion. The SFTTrainer is configured for 1 epoch with a learning rate of 2e-4 and an effective batch size of 16 via gradient accumulation. To enable direct comparison with the Part D prompting experiments, evaluation is performed on the same 400-document stratified subset of 20 documents per class. QLoRA fine-tuning achieves 63.0% accuracy and macro-F1 of 0.681, a 14 percentage point accuracy gain over the best prompting result on the same evaluation set, confirming that

even minimal gradient-based adaptation substantially outperforms inference-only prompting. However, QLoRA does not close the gap with full BERT fine-tuning: MLM-adapted BERT achieves 73.6% accuracy updating 109M parameters across 3 epochs with a purpose-built classification objective, whereas QLoRA updates 1% of parameters for 1 epoch on a generative text completion task. The 17.5% unparseable rate further suppresses the accuracy figure, as responses failing the three-tier label matcher are recorded as misclassifications regardless of semantic proximity to the correct label.

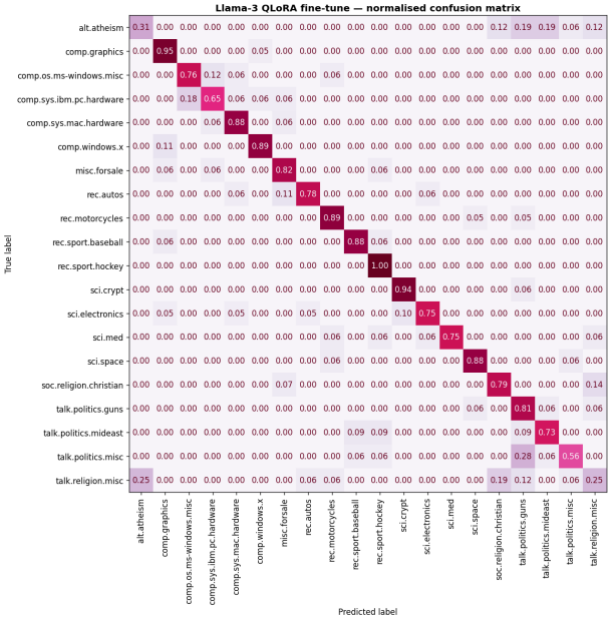

Figure 12 - Normalised Confusion Matrix, Llama-3 QLoRA Fine-Tune

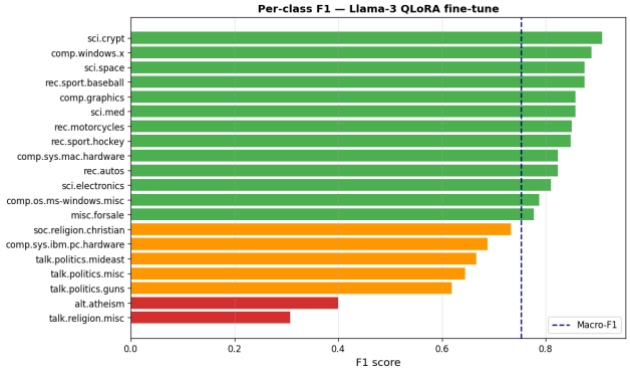

Figure 13 - Per-Class F1 Score, Llama-3 QLoRA Fine-Tune

Figure 12 depicts the confusion matrix for the fine-tuned model using QLoRA which is the most visually striking confusion matrix of all five. Several things stand out immediately:

The diagonal is strong across most technical and recreational categories, rec.sport.hockey achieves a perfect 1.00, comp.graphics hits 0.95, sci.crypt hits 0.94, and rec.motorcycles, comp.windows.x, and comp.sys.mac.hardware all sit at 0.88–0.89. These are substantially better than the same classes in BERT fine-tuning or TF-IDF. QLoRA has clearly learned that these categories have distinctive enough vocabulary to classify almost perfectly. However, the religion/politics cluster remains the model's Achilles' heel, alt.atheism drops to just 0.31, with mass confusion spreading to soc.religion.christian (0.12), talk.politics.guns (0.19), talk.politics.misc (0.06), and talk.religion.misc (0.12). talk.religion.misc itself scores only 0.25. These are the worst per-class results across the entire project. The model has essentially given up on cleanly separating these categories, which pulls the overall accuracy down to 63% despite excellent performance elsewhere. The 17.5% unparseable rate is also visible indirectly, some of the confusion may reflect the model generating outputs that could not be mapped to a valid label, counted as misclassifications. This is notably higher than Part D's 3.5% and is a methodological limitation worth flagging: the instruction-following format did not fully constrain Llama-3's output, suggesting the SFT training did not sufficiently enforce rigid label output.

Figure 13, depicts a horizontal bar chart ranking all 20 classes by F1 score for the QLoRA model, colour-coded green/orange/red relative to the macro-F1 (0.681, dashed line). The bimodal nature of this chart is immediately visible; 13 classes sit comfortably in the green zone (above 0.77), while the bottom 7 cluster below 0.75 with a sharp drop to alt.atheism (~0.40) and talk.religion.misc (~0.31). This is a more extreme version of the pattern seen in Part A's TF-IDF chart, where the spread was more gradual. QLoRA has created a model that is very good at most categories but catastrophically poor at the religion/politics cluster, the macro-F1 (0.681) masks this by averaging across all 20 classes. QLoRA's overall accuracy (63%) underestimates its capability on well-defined categories, while the high unparseable rate and religion cluster confusion pull the aggregate metric down, the category ordering is almost inverted. Sci.crypt and comp.windows top the QLoRA chart but sat in the middle of the TF-IDF chart; however, talk.religion.misc is ranked last in both, showing that this is a difficult topic to class.

III. Results

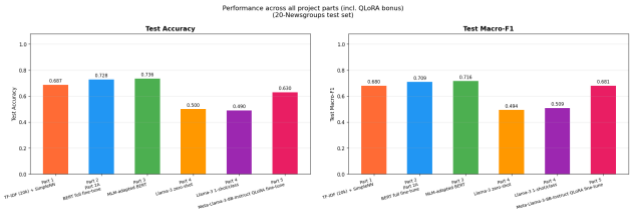

Figure 14 - Full Cross-Project Performance Comparison Including QLoRA

Figure 14 is the full accuracy of the models and Table 2 presents the full performance (all experiments, test accuracy and macro-F1). The gap between W2V 1 epoch (14.4%) and W2V 20 epochs (58.8%) is the largest single improvement in the study, a reminder that representation quality matters more than architecture choice in early-stage models. Note that TF-IDF (68.7%) outperforms Word2Vec and GloVe despite being a decades-old method which highlights the importance of matching the method to the task's structure The frozen BERT (38.7%) result is analytically important: it establishes that pre-trained representations are not inherently task-adapted; fine-tuning is not optional.

MLM-adapted BERT achieves the highest performance across all experiments at 73.6% accuracy and macro-F1 of 0.716 (Figure 14), marginally outperforming plain BERT fine-tuning (72.8%, 0.709) by 0.8 percentage points in accuracy and 0.007 in macro-F1. Given that both models are evaluated on the same fixed test set, this difference cannot be subjected to significance testing without repeated trials across multiple splits, and should therefore be interpreted cautiously, the gain is consistent but modest, and may partly reflect the specific composition of the test partition rather than a robust generalisation advantage. The more analytically important finding of Part D is not the absolute accuracy figures but what they reveal about the relationship between model scale and task performance. Llama-3-8B, with approximately 73 times more parameters than BERT-base, achieves only 50.0% accuracy under zero-shot conditions and 49.0% under 1-shot conditions, both well below TF-IDF (68.7%), a method requiring no neural network whatsoever. Evaluating these results purely by the numbers understates the significance: the zero-shot and few-shot conditions involve a model of substantially greater architectural complexity, operating without any gradient-based adaptation to the target task, being asked to output one of 20 specific label strings through prompt design alone. The per-class F1 chart for the Llama-3 few-shot condition and the corresponding confusion matrix (Figure 10) show dispersed, inconsistent errors across categories that fine-tuned BERT resolves cleanly, a pattern that reflects the absence of task-specific training rather than a fundamental incapacity of the model.

Table 2 - Model Performance Comparison

Experiment	Test Acc	Macro-F1
Word2Vec (1 epoch)	14.4%	0.107
Word2Vec (20 epochs)	58.8%	0.571
GloVe pre-trained	57.2%	0.552
TF-IDF (20k)	68.7%	0.680
BERT linear probe (frozen)	38.7%	0.334
BERT partial freeze	71.0%	0.688
BERT full fine-tune	72.8%	0.709
MLM-adapted BERT	**73.6%**	**0.716**
Llama-3 zero-shot	50.0%	0.494
Llama-3 1-shot/class	49.0%	0.509
Llama-3 QLoRA fine-tune	63.0%	0.681

Across all experiments, MLM-adapted BERT represents the performance ceiling of this project, achieving 73.6% accuracy and macro-F1 of 0.716. The result is the product of three compounding advantages: the representational depth of a bidirectional transformer architecture, the task-specific adaptation provided by full fine-tuning of all 109M parameters, and the domain alignment introduced by continued MLM pre-training on the newsgroup corpus prior to classification. No other configuration in this project combines all three: Part A lacks the architecture, Part B lacks the domain pre-training, and Parts D and E lack the full fine-tuning. That the margin over plain BERT fine-tuning (72.8%) is narrow rather than large is itself informative: the newsgroup corpus is sufficiently small that domain adaptation yields diminishing returns, and the classification signal available through standard fine-tuning already captures most of the discriminative structure in the data. The per-class F1 chart and confusion matrix for Part C confirm that even the best model in this project does not fully resolve the religion and politics cluster, a finding consistent across every method tested, suggesting the residual error reflects genuine semantic overlap in the data rather than a limitation of any particular.

IV. Discussion

A. Strengths

Controlled experimental design: same dataset split, same evaluation metric (macro-F1), same preprocessing pipeline across Parts A–C; this makes comparisons meaningful

The progression is deliberately pedagogical: each part adds one variable (representation, then model scale, then pre-training objective, then inference paradigm)

The additional experiment (GloVe) tests a genuine hypothesis with a clear prediction, and the result (hypothesis not supported) is itself informative

The fill-mask qualitative check in Part C provides interpretable evidence of domain adaptation beyond the accuracy numbers

B. Weaknesses

The QLoRA experiment (Part E) uses a 1B model rather than the 8B used in Part D, making cross-part comparisons less clean; this is a hardware constraint, but should be stated as a limitation

Few-shot in Part D tests only k=1 per class; k=3 or k=5 may have shown a clearer improvement trend

No statistical significance testing: with a single train/test split, it is not possible to determine whether the 0.8% gap between Part B and Part C is robust or within noise

The simple neural network architecture is deliberately constrained; it is not representative of what a feedforward network can achieve with more capacity

C. Lessons Learned

Representation quality is the dominant factor for shallow networks: the architecture (two linear layers + ReLU) stays fixed, but performance ranges from 14.4% to 68.7% depending solely on the input representation

Domain-specific vocabulary rewards in-domain pre-training: the marginal gain of Part C over Part B reflects BERT learning Usenet-specific language patterns that were absent from its original Wikipedia + BookCorpus pre-training

LLM prompting without fine-tuning is not a free lunch: the Llama-3 results challenge the assumption that larger models are always better; task-specific training remains critical for structured, label-constrained classification tasks

Parameter-efficient fine-tuning (QLoRA) opens the door to fine-tuning models that would otherwise be intractable, but the comparison with full fine-tuning of smaller encoder models is not straightforward

V. Conclusion

This paper investigated the progression of text classification on the 20-Newsgroups benchmark, evaluating five methodological paradigms: static word embeddings with shallow feedforward networks, full and partial BERT fine-tuning, task-adaptive MLM pre-training, large language

model prompting with Llama-3, and parameter-efficient fine-tuning via QLoRA.

The experimental results demonstrate a clear performance hierarchy aligned with representational richness and task-specific adaptation. Among static embedding methods, TF-IDF achieved the highest accuracy (68.7%, macro-F1 0.680), outperforming both Word2Vec and GloVe due to the heavily lexical nature of the 20-Newsgroups corpus, where rare, domain-specific tokens carry strong categorical signal that mean-pooled dense embeddings dilute. BERT fine-tuning substantially exceeded all static-embedding baselines, with full fine-tuning reaching 72.8% accuracy (macro-F1 0.709). The frozen linear probe result (38.7%) demonstrated that pre-trained representations, while rich, are not inherently task-adapted and require fine-tuning to realise their representational potential. Task-adaptive MLM pre-training on the 20-Newsgroups corpus prior to classification fine-tuning yielded the best overall result, 73.6% accuracy and macro-F1 of 0.716, confirming that narrowing the domain gap between general pre-training data and the target corpus produces measurable gains. Llama-3-8B, despite its parameter scale, achieved 50.0% accuracy under zero-shot prompting and 49.0% under one-shot-per-class prompting, underscoring that model size alone does not guarantee strong performance on structured, label-constrained classification tasks in the absence of task-specific training. QLoRA fine-tuning of Llama-3.2-1B partially recovered this deficit, reaching 63.0% accuracy, though the use of a smaller base model than Part D limits the directness of the comparison.

The primary contribution of this study is a controlled, end-to-end comparison across the dominant NLP paradigms of the past decade on a single canonical benchmark, enabling direct assessment of the representational and computational trade-offs at each step. The results reaffirm that domain-adapted encoder models remain highly competitive for structured multi-class classification, even against significantly larger generative models evaluated under prompting conditions.

Several limitations constrain the conclusions drawn. The QLoRA experiment used a 1B-parameter model due to hardware constraints, rather than the 8B model used in prompting experiments, reducing the comparability of Parts 4 and 5. Few-shot evaluation was restricted to k=1 per class; higher values of k may reveal a stronger benefit curve. No statistical significance testing was applied, given a single train/test split, the 0.8 percentage point gap between plain fine-tuning and MLM-adapted BERT cannot be confirmed as robust rather than noise. Finally, the shallow feedforward architecture in Part A was deliberately constrained and is not representative of what feedforward networks can achieve at greater capacity.

Future work should explore higher values of k in few-shot prompting and assess whether the gains plateau, apply significance testing over multiple train/test splits, and investigate whether full fine-tuning of Llama-3-8B via QLoRA closes the gap with encoder-based models. Extending domain-adaptive pre-training to a broader Usenet corpus and combining DAPT with retrieval-augmented generation represent promising directions for further improving classification performance on domain-specific corpora.

AI Disclosure

This project was developed with the assistance of an AI coding and writing assistant (Claude, made by Anthropic). In line with academic guidance on disclosure of generative-AI tool usage, the following describes the role AI played in this work.

AI assistance was used throughout the development of the codebase for: Debugging and error resolution: diagnosing Python runtime errors, Docker and Cloud Build failures, deployment issues on Cloud Run, and shell / gcloud command misconfigurations. As well as architectural, design and code feedback. All AI-generated code was reviewed, tested, and integrated by the author. All cloud deployments and live runs were executed and verified by the author. Final design decisions and responsibility for the working system rest with the author.

AI assistance was used in preparing the written report for: grammar, spelling, proof-reading, and structural feedback, including critiques and recommendations. The substantive arguments, analytical conclusions, and selection of evidence in the report are the author's own.

References

[1] A. Vaswani *et al.*, 'Attention Is All You Need', *CoRR*, vol. abs/1706.03762, 2017, [Online]. Available: http://arxiv.org/abs/1706.03762

[2] T. B. Brown *et al.*, 'Language Models are Few-Shot Learners', *CoRR*, vol. abs/2005.14165, 2020, [Online]. Available: https://arxiv.org/abs/2005.14165

[3] 'GPT-4', OpenAI. Accessed: Jun. 05, 2026. [Online]. Available: https://openai.com/index/gpt-4-research/

[4] K. Lang, 'NewsWeeder: learning to filter netnews', in *Proceedings of the Twelfth International Conference on International Conference on Machine Learning*, in ICML'95. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1995, pp. 331–339.

[5] T. Mikolov, K. Chen, G. Corrado, and J. Dean, 'Efficient Estimation of Word Representations in Vector Space'. 2013. [Online]. Available: https://arxiv.org/abs/1301.3781

[6] J. Alammar, 'The Illustrated Word2vec'. Accessed: Jun. 05, 2026. [Online]. Available: https://jalammar.github.io/illustrated-word2vec/

[7] J. Pennington, R. Socher, and C. Manning, 'Glove: Global Vectors for Word Representation', in *EMNLP*, Jan. 2014, pp. 1532–1543. doi: 10.3115/v1/D14-1162.

[8] 'TfidfVectorizer', scikit-learn. Accessed: Jun. 06, 2026. [Online]. Available: https://scikit-learn/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

[9] J. Alammar and M. Grootendorst, *Hands-on large language models: language understanding and generation*. Beijing Boston Farnham Sebastopol Tokyo: O'Reilly, 2024.

[10] J. Barnard, 'What is Embedding? | IBM'. Accessed: Jun. 06, 2026. [Online]. Available: https://www.ibm.com/think/topics/embedding

[11] W. C. Huyen and K. Li, *Designing Machine Learning Systems*. Accessed: Jun. 05, 2026. [Online]. Available: https://learning.oreilly.com/library/view/designing-machine-learning/9781098107956/

[12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding', in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds, Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186. doi: 10.18653/v1/N19-1423.

[13] S. Gururangan *et al.*, 'Don't Stop Pretraining: Adapt Language Models to Domains and Tasks', in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, Eds, Online: Association for Computational Linguistics, Jul. 2020, pp. 8342–8360. doi: 10.18653/v1/2020.acl-main.740.

[14] 'Tokenization algorithms · Hugging Face'. Accessed: Jun. 05, 2026. [Online]. Available: https://huggingface.co/docs/transformers/tokenizer_summary

[15] T. Dettmers, A. Holtzman, A. Pagnoni, and L. Zettlemoyer, 'QLoRA: Efficient Finetuning of Quantized LLMs', Jan. 2023, pp. 10088–10115. doi: 10.52202/075280-0441.

[16] 'Fine-Tuning Llama 3.1 for Text Classification'. Accessed: Jun. 05, 2026. [Online]. Available: https://www.datacamp.com/tutorial/fine-tuning-llama-3-1

[17] S. M, 'Pooling in Embedding', Medium. Accessed: Jun. 06, 2026. [Online]. Available: https://medium.com/@suvasism/pooling-in-embedding-16781aacfb12